

Функции в SCSS

Что такое функции в SCSS?

Функции в SCSS позволяют определять сложные операции, которые можно повторно использовать в таблице стилей. В отличие от миксинов, которые генерируют CSS-код, функции возвращают значение, которое можно использовать в свойствах CSS.

Встроенные функции

SCSS предоставляет множество встроенных функций для работы с различными типами данных.

Функции для работы с цветами

```
$base-color: #3498db;

.element {
  // Осветление цвета
  background-color: lighten($base-color, 10%); // #5faee3

  // Затемнение цвета
  border-color: darken($base-color, 10%); // #217dbb

  // Изменение насыщенности
  color: saturate($base-color, 20%); // #1a8be9

  // Уменьшение насыщенности
  background-color: desaturate($base-color, 20%); // #4e94c3

  // Изменение прозрачности
  border-color: rgba($base-color, 0.5); // rgba(52, 152, 219, 0.5)

  // Смешивание цветов
  color: mix($base-color, #e74c3c, 30%); // #d44e3e
}
```

Функции для работы с числами

```
.element {
  // Округление
  width: round(4.6px); // 5px

  // Округление вниз
  height: floor(4.6px); // 4px

  // Округление вверх
```

```
padding: ceil(4.2px); // 5px

// Абсолютное значение
margin: abs(-10px); // 10px

// Минимальное значение
font-size: min(16px, 14px, 18px); // 14px

// Максимальное значение
line-height: max(1.2, 1.5, 1.3); // 1.5

// Случайное число
z-index: random(100); // случайное число от 1 до 100

// Процентное соотношение
width: percentage(0.8); // 80%
}
```

Функции для работы со строками

```
$font: 'Roboto';
$path: 'assets/images/';

.element {
  // Объединение строк
  font-family: #{$font}, sans-serif;

  // Добавление кавычек
  content: quote>Hello);

  // Удаление кавычек
  font-family: unquote($font);

  // Получение длины строки
  z-index: str-length($font); // 6

  // Получение подстроки
  content: str-slice($font, 1, 3); // 'Rob'

  // Преобразование в нижний регистр
  text-transform: to-lower-case('HELLO'); // 'hello'

  // Преобразование в верхний регистр
  text-transform: to-upper-case('hello'); // 'HELLO'

  // Вставка строки
  background-image: url(#{$path}logo.png);
}
```

Функции для работы со списками

```
$sizes: 10px 20px 30px 40px;
$colors: red, green, blue;

.element {
  // Получение элемента по индексу (индексация начинается с 1)
  padding: nth($sizes, 2); // 20px

  // Получение длины списка
  z-index: length($sizes); // 4

  // Получение первого элемента
  margin-top: first-value($sizes); // 10px (Sass 1.35.0+)

  // Получение последнего элемента
  margin-bottom: last-value($sizes); // 40px (Sass 1.35.0+)

  // Объединение списков
  $combined: join($sizes, $colors);

  // Добавление элемента в список
  $new-sizes: append($sizes, 50px);

  // Поиск индекса элемента
  $index: index($colors, green); // 2
}
```

Функции для проверки типов

```
$color: #3498db;
$number: 42;
$string: 'Hello';
$list: 10px 20px 30px;
$map: (key1: value1, key2: value2);
$bool: true;

@debug type-of($color); // color
@debug type-of($number); // number
@debug type-of($string); // string
@debug type-of($list); // list
@debug type-of($map); // map
@debug type-of($bool); // bool

// Проверка типа
@if type-of($color) == color {
  // Действия для цвета
}
```

Пользовательские функции

Определение функции

Функции в SCSS определяются с помощью директивы `@function`:

```
@function function-name($parameter1, $parameter2, ...) {  
  // Логика функции  
  @return $value;  
}
```

Простые примеры функций

```
// Функция для преобразования пикселей в em  
@function px-to-em($px, $base-font-size: 16px) {  
  @return ($px / $base-font-size) * 1em;  
}  
  
// Функция для вычисления ширины колонки в сетке  
@function column-width($columns, $total-columns: 12, $gutter: 20px, $container-width: 1200px) {  
  $total-gutter-width: ($total-columns - 1) * $gutter;  
  $total-column-width: $container-width - $total-gutter-width;  
  $single-column-width: $total-column-width / $total-columns;  
  
  @return ($single-column-width * $columns) + ($gutter * ($columns - 1));  
}  
  
// Использование функций  
.element {  
  font-size: px-to-em(24px); // 1.5em  
  width: column-width(4); // ширина для 4 колонок из 12  
}
```

Функции с условной логикой

```
// Функция для выбора цвета текста в зависимости от фона  
@function text-color-for-bg($bg-color) {  
  $brightness: (red($bg-color) * 299 + green($bg-color) * 587 + blue($bg-color) * 114) / 1000;  
  
  @if $brightness > 128 {  
    @return #000; // Темный текст для светлого фона  
  } @else {  
    @return #fff; // Светлый текст для темного фона  
  }  
}
```

```
// Функция для получения цвета из палитры
@function color($name, $variant: 'base') {
  $colors: (
    primary: (
      base: #3498db,
      light: #5faee3,
      dark: #217dbb
    ),
    secondary: (
      base: #2ecc71,
      light: #54d98c,
      dark: #25a25a
    ),
    danger: (
      base: #e74c3c,
      light: #ed7669,
      dark: #d62c1a
    )
  );

  @if not map-has-key($colors, $name) {
    @error "Цвет '#{ $name }' не найден в палитре.";
  }

  $color-variants: map-get($colors, $name);

  @if not map-has-key($color-variants, $variant) {
    @error "Вариант '#{ $variant }' для цвета '#{ $name }' не найден.";
  }

  @return map-get($color-variants, $variant);
}

// Использование функций
.button {
  background-color: color(primary);
  color: text-color-for-bg(color(primary)); // #fff
}

.alert {
  background-color: color(danger, 'light');
  color: text-color-for-bg(color(danger, 'light')); // #000
}
```

Рекурсивные функции

```
// Функция для вычисления факториала
@function factorial($n) {
  @if $n == 0 or $n == 1 {
```

```

    @return 1;
  } @else {
    @return $n * factorial($n - 1);
  }
}

// Функция для генерации последовательности Фибоначчи
@function fibonacci($n) {
  @if $n == 1 {
    @return 0;
  } @else if $n == 2 {
    @return 1;
  } @else {
    @return fibonacci($n - 1) + fibonacci($n - 2);
  }
}

// Использование
.element {
  z-index: factorial(5); // 120
  padding: fibonacci(7) * 1px; // 8px
}

```

Функции vs. Миксины

Важно понимать разницу между функциями и миксинами в SCSS:

- **Функции** возвращают значение, которое используется как часть значения свойства или переменной
- **Миксины** генерируют CSS-код, который включается в таблицу стилей

```

// Функция - возвращает значение
@function double($value) {
  @return $value * 2;
}

// Миксин - генерирует CSS-код
@mixin double-margin($value) {
  margin-top: $value;
  margin-bottom: $value;
}

.element {
  // Использование функции - возвращает значение
  width: double(100px); // 200px

  // Использование миксина - генерирует CSS-свойства
  @include double-margin(20px);
}

```

Лучшие практики использования функций

1. Используйте функции для вычислений

- Функции идеально подходят для математических операций и преобразований значений

2. Давайте функциям ясные, описательные имена

- Имя должно отражать, что делает функция и что она возвращает

3. Документируйте параметры функций

- Добавляйте комментарии, объясняющие назначение каждого параметра

4. Организуйте функции в отдельных файлах

- Храните функции в отдельном файле (например, `_functions.scss`)

5. Используйте параметры со значениями по умолчанию

- Это делает функции более гибкими и удобными в использовании

6. Проверяйте входные данные

- Используйте директивы `@error` и `@warn` для проверки входных параметров

```
@function color($name) {  
  $colors: (  
    primary: #3498db,  
    secondary: #2ecc71,  
    danger: #e74c3c  
  );  
  
  @if not map-has-key($colors, $name) {  
    @error "Цвет '#{ $name }' не найден в палитре. Доступные цвета: #{map-keys($colors)}.";  
  }  
  
  @return map-get($colors, $name);  
}
```

Практические примеры использования функций

Система типографики

```
// Шкала типографики  
$type-scale: 1.25;  
$base-font-size: 16px;  
  
@function type-scale($level) {
```

```

    @return $base-font-size * pow($type-scale, $level);
}

// Вспомогательная функция для возведения в степень
@function pow($base, $exponent) {
    $result: 1;

    @if $exponent > 0 {
        @for $i from 1 through $exponent {
            $result: $result * $base;
        }
    } @else if $exponent < 0 {
        @for $i from 1 through -$exponent {
            $result: $result / $base;
        }
    }

    @return $result;
}

// Использование
h1 {
    font-size: type-scale(4); // ~39.06px
}

h2 {
    font-size: type-scale(3); // ~31.25px
}

h3 {
    font-size: type-scale(2); // ~25px
}

h4 {
    font-size: type-scale(1); // ~20px
}

p {
    font-size: type-scale(0); // 16px
}

small {
    font-size: type-scale(-1); // ~12.8px
}

```

Система цветов

```

// Функция для создания цветовой палитры
@function create-palette($base-color, $steps: 5, $step-size: 10%) {
    $palette: ();

```



```

@for $i from -$steps through $steps {
  @if $i < 0 {
    $key: 'darken' + abs($i);
    $value: darken($base-color, abs($i) * $step-size);
  } @else if $i > 0 {
    $key: 'lighten' + $i;
    $value: lighten($base-color, $i * $step-size);
  } @else {
    $key: 'base';
    $value: $base-color;
  }

  $palette: map-merge($palette, ($key: $value));
}

@return $palette;
}

// Функция для получения цвета из палитры
@function palette-color($palette, $variant: 'base') {
  @if not map-has-key($palette, $variant) {
    @error "Вариант '#{ $variant }' не найден в палитре.";
  }

  @return map-get($palette, $variant);
}

// Использование
$primary-palette: create-palette(#3498db);
$secondary-palette: create-palette(#2ecc71);

.button {
  background-color: palette-color($primary-palette, 'base');
  border-color: palette-color($primary-palette, 'darken2');

  &:hover {
    background-color: palette-color($primary-palette, 'darken1');
  }

  &.secondary {
    background-color: palette-color($secondary-palette, 'base');
    border-color: palette-color($secondary-palette, 'darken2');

    &:hover {
      background-color: palette-color($secondary-palette, 'darken1');
    }
  }
}

```

Система отзывчивого дизайна

```

$breakpoints: (
  xs: 0,
  sm: 576px,
  md: 768px,
  lg: 992px,
  xl: 1200px
);

// Функция для получения значения точки останова
@function breakpoint($name) {
  @if not map-has-key($breakpoints, $name) {
    @error "Точка останова '#{ $name }' не найдена. Доступные точки: #{map-keys($breakpoints)}.";
  }

  @return map-get($breakpoints, $name);
}

// Миксин для медиа-запросов, использующий функцию
@mixin media-up($name) {
  @media (min-width: breakpoint($name)) {
    @content;
  }
}

// Использование
.container {
  width: 100%;

  @include media-up(sm) {
    width: 540px;
  }

  @include media-up(md) {
    width: 720px;
  }

  @include media-up(lg) {
    width: 960px;
  }

  @include media-up(xl) {
    width: 1140px;
  }
}

```

Заключение

Функции в SCSS - это мощный инструмент для создания динамических, вычисляемых значений в ваших стилях. Они особенно полезны для работы с цветами, размерами, математическими

вычислениями и другими операциями, которые требуют логики. Правильное использование функций может значительно улучшить организацию вашего кода, уменьшить дублирование и сделать стили более гибкими и поддерживаемыми.